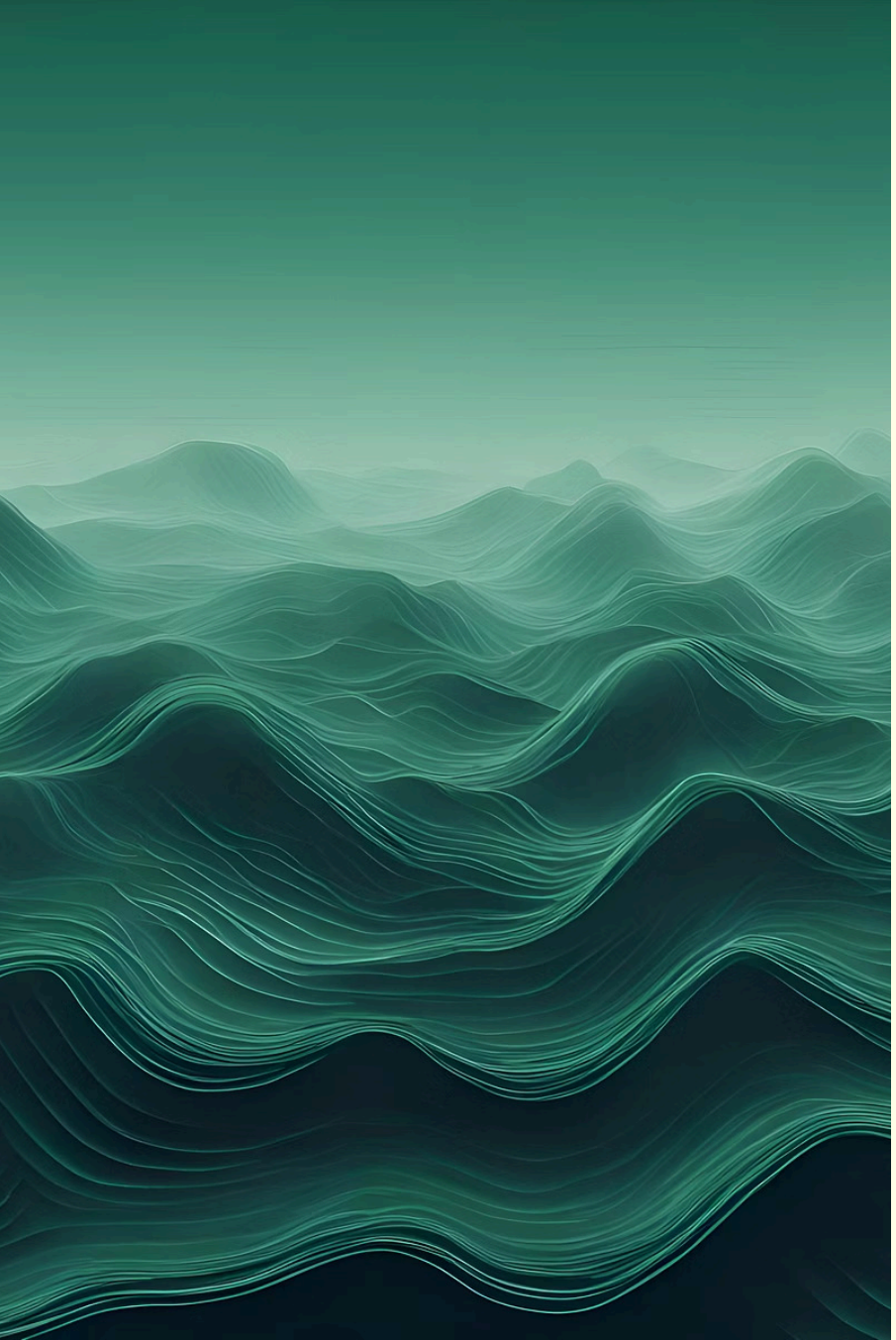


Day 7: Model Building, Hyperparameter Tuning & Model Evaluation - Teaching Session

Presented by : Sinu Maria Jeeson

Made with GAMMA

Typical Machine Learning Pipeline for Text Analysis

Each stage in this pipeline transforms data while introducing opportunities to optimize performance. Understanding the complete flow is essential for diagnosing issues and improving results.

Raw Text Data

Unstructured text from reviews, social media, documents, or other sources

Text Preprocessing

Tokenization, stopword removal, stemming/lemmatization, and data cleaning operations

Feature Extraction

Converting cleaned text into numerical vectors using TF-IDF, embeddings, or other techniques

Model Building

Training classifiers like Logistic Regression, SVM, or Random Forest on extracted features

Model Evaluation

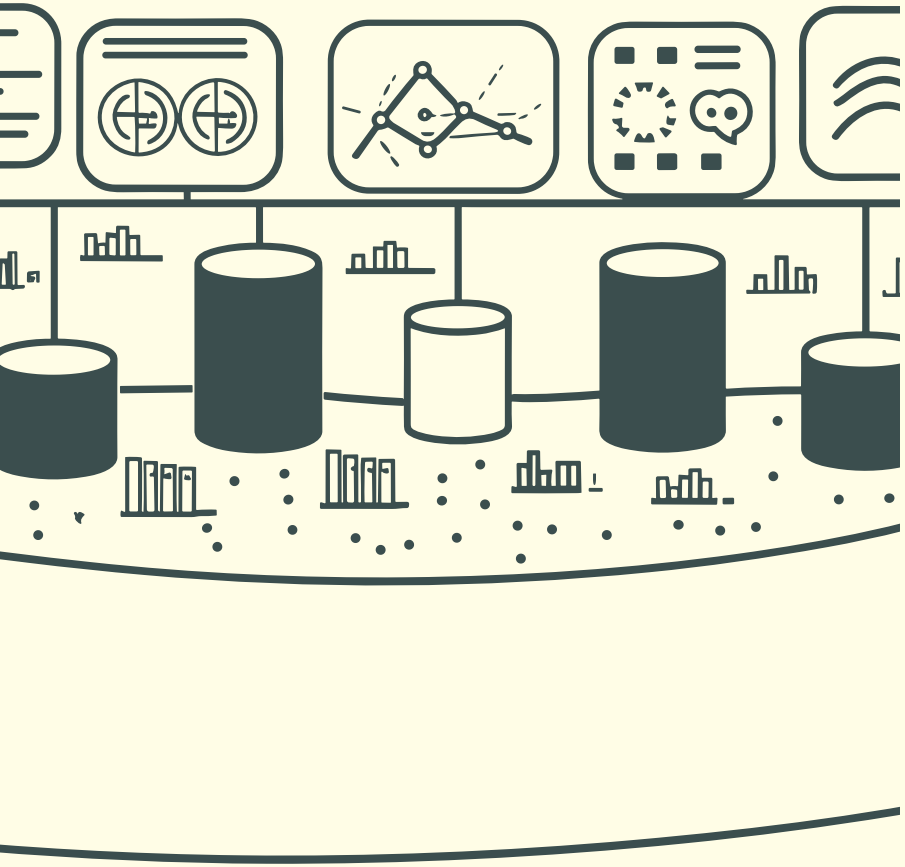
Assessing performance using confusion matrices, F1 scores, accuracy, and other metrics

Model Improvement

Applying cross-validation, hyperparameter tuning, and ensemble methods to boost performance

Critical insight: Each preprocessing decision and modeling choice cascades through subsequent stages. Poor text cleaning amplifies feature extraction errors. Suboptimal hyperparameters limit model capacity. This interconnectedness makes pipeline awareness essential.

Made with GAMMA



Model Building for Sentiment Analysis using TF-IDF

This mentoring session focuses on converting raw product reviews into actionable machine learning features using TF-IDF vectorization. We'll explore the complete workflow from text preprocessing to model evaluation, with emphasis on practical implementation and interpretation.

Made with GAMMA

The Complete ML Workflow

01

Text Data

Raw product reviews with sentiment labels

02

Text Preprocessing

Cleaning, normalization, and tokenization

03

TF-IDF Feature Extraction

Converting text to numerical vectors

04

Model Training

Learning patterns from features

05

Model Evaluation

Measuring performance with metrics

06

Improve Preprocessing

Iterating based on model feedback

Feature engineering often improves performance more than switching models. The preprocessing pipeline directly determines what patterns your model can learn.

Made with GAMMA

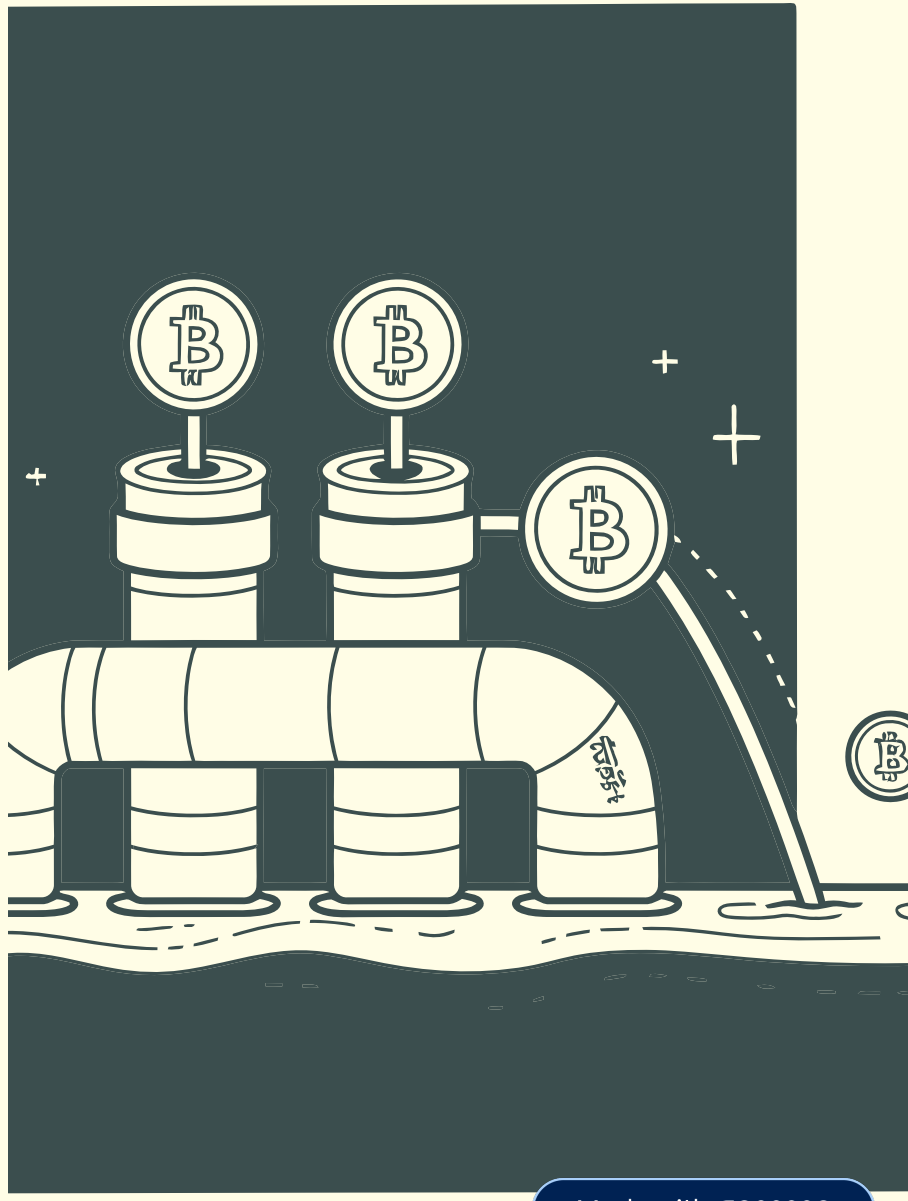
Why Text Preprocessing is Essential

The Challenge

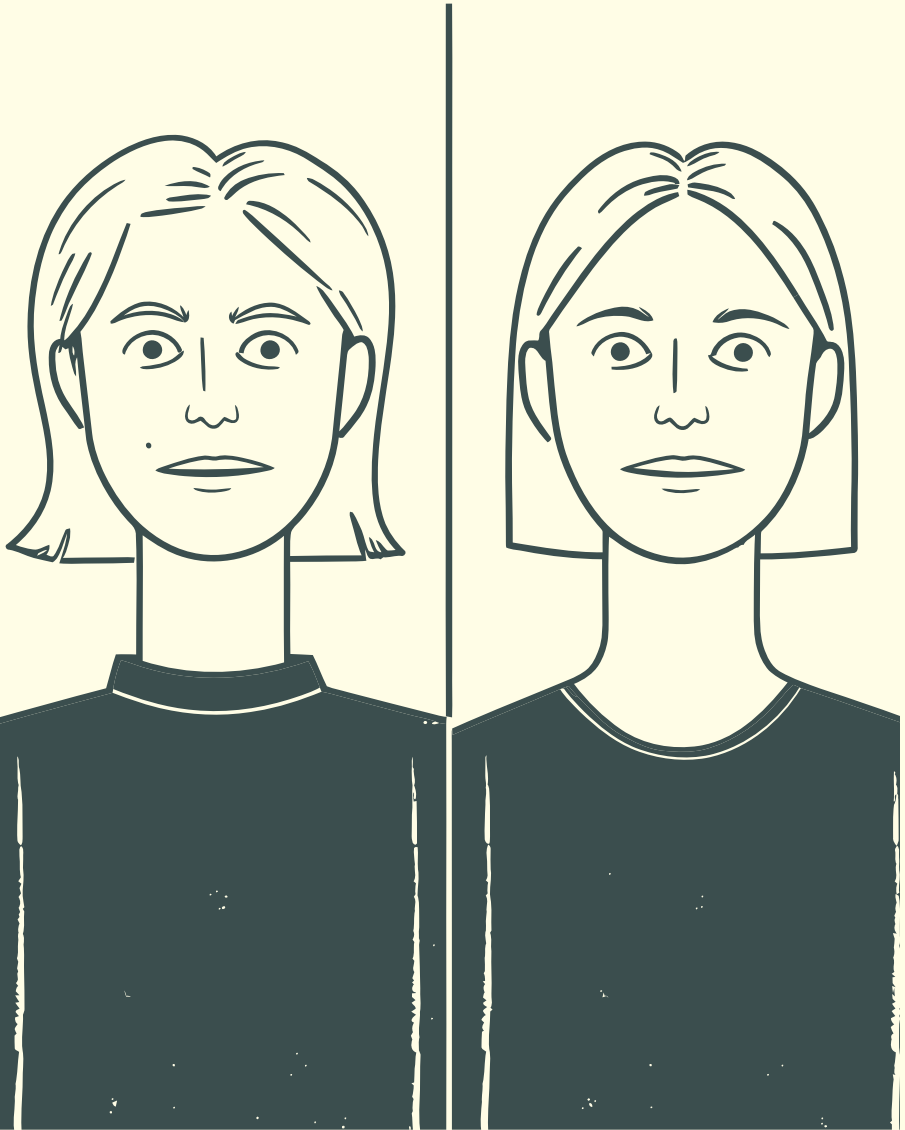
Machine learning models cannot process raw text directly. Every algorithm requires numerical input features. Text preprocessing bridges this gap by transforming unstructured text into a standardized numerical format that models can understand and learn from.

Key Preprocessing Steps

- Lowercasing all text for consistency
- Removing punctuation and special characters
- Tokenization into individual words
- Stopword removal to reduce noise
- Stemming or lemmatization for normalization



Made with GAMMA



Preprocessing Transformation Example

1

Raw Review

"THIS PRODUCT IS AMAZING!!! Works perfectly."

2

After Preprocessing

["product", "amazing", "works", "perfectly"]

The preprocessing pipeline systematically transforms the original text. First, it converts to lowercase, then removes punctuation and exclamation marks. The text is split into tokens, stopwords like "is" and "this" are filtered out, and finally, words may be stemmed to their root forms.

Made with GAMMA

TF-IDF: Term Frequency-Inverse Document Frequency

TF-IDF transforms text into weighted numerical features that reflect word importance. The core intuition is simple: common words should have lower importance while rare, meaningful words should have higher weights.

Term Frequency (TF)

$$TF(t,d) = \frac{\text{frequency of word } t \text{ in document } d}{\text{total words in document}}$$

Measures how often a word appears in a specific document

Inverse Document Frequency (IDF)

$$IDF(t) = \log \frac{N}{df(t)}$$

N = total documents, df(t) = documents containing word t

Logarithm prevents extremely common words from dominating

TF-IDF Score

$$TF\text{-}IDF(t,d) = TF(t,d) \times IDF(t)$$

Combines both measures to weight word importance

Made with GAMMA

TF-IDF Example with 3 Documents

Document 1

"The product is good"

Document 2

"The product is bad"

Document 3

"This product is very good"

The word **product** appears in all three documents, so it receives a low IDF score and low overall importance. The word **bad** appears in only one document, receiving a high IDF score and significant weight. Rare sentiment words like "bad" and "excellent" get higher weights, helping the model focus on meaningful indicators.



Made with GAMMA

Key TF-IDF Implementation Parameters

```
vectorizer = TfidfVectorizer(
    ngram_range=(1,2),
    min_df=5
)
```

ngram_range = (1,2)

Generates both unigrams (single words) and bigrams (word pairs). Bigrams like "not good" and "very bad" are crucial for sentiment detection because they capture negation and intensity patterns that single words miss.

min_df = 5

Ignore words that appear in fewer than 5 documents. This reduces noise in the feature space by eliminating extremely rare terms that likely won't help with prediction and could cause overfitting.

Made with GAMMA

Logistic Regression for Sentiment Classification

Logistic Regression calculates a weighted sum of TF-IDF features and transforms it into probabilities using the sigmoid function. The model learns which words indicate positive or negative sentiment through coefficient values.

```
model = LogisticRegression()
model.fit(X_train, y_train)
```



Interpreting Coefficients

```
good      → +1.45
excellent → +2.10
bad       → -1.85
```

Positive coefficients indicate positive sentiment words while negative coefficients indicate negative sentiment words. The magnitude shows relative importance.

Made with GAMMA

Random Forest and Model Evaluation

Random Forest Concept

Builds many decision trees and combines predictions through majority voting. Each tree learns different patterns, making the ensemble more robust.

Evaluation Metrics

Accuracy: Overall correctness

Precision: Quality of positives

Recall: Coverage of actual positives

F1 Score: Balance between precision and recall

Classification Report Example

	precision	recall	f1-score
positive	0.89	0.92	0.90
neutral	0.78	0.70	0.74
negative	0.91	0.88	0.89

The report reveals the model performs well on positive and negative classes but struggles with neutral reviews, indicating potential preprocessing improvements.

Made with GAMMA

Diagnosing Errors and Improving Performance

Confusion Matrix Analysis

	Predicted Positive	Neutral	Negative
Actual Positive	80	12	3
Actual Neutral	10	40	15
Actual Negative	2	8	70

Many positives predicted as neutral reveals the model struggles with moderate sentiment. Missing phrase patterns like "not bad" or "kind of good" suggests adding bigrams would help.

Key Insights

- Model results reveal preprocessing weaknesses
- Include bigrams when negation patterns exist
- Adjust TF-IDF parameters to capture important words
- Scaling matters for Logistic Regression, SVM, KNN but not tree-based models

Final Takeaway: Better feature engineering and preprocessing often improve model performance more than switching algorithms. Iterate on preprocessing based on model feedback.

Made with GAMMA

Case Example 1: The "Not Good" Problem

The Confusion

After evaluating your sentiment model, you notice a pattern in the confusion matrix:

- 80 positive reviews correctly classified
- 15 positive reviews misclassified as neutral
- 10 negative reviews misclassified as positive

What's Happening

Negative reviews containing phrases like "not good", "not great", and "not worth it" are being predicted as positive.

Why? If your TF-IDF vectorizer uses only unigrams (single words), the model sees "not good" as two separate tokens: ["not", "good"].

The word "good" carries strong positive weight, so the model predicts positive sentiment despite the negation.



1

Problem Signal

Frequent misclassification involving negation phrases

2

Diagnosis

Missing linguistic patterns in preprocessing

3

Solution

Use bigrams in TF-IDF

Made with GAMMA

Implementing Bigram Features

Before: Unigrams Only

`ngram_range = (1, 1)`

Features: "not", "good", "great", "excellent"

Problem: Loses negation context

After: Unigrams + Bigrams

`ngram_range = (1, 2)`

Features: "not", "good", "not good", "great", "not great"

Solution: Captures negation phrases

By expanding the n-gram range, the feature "not good" now carries negative sentiment independently of the word "good". This simple preprocessing change can significantly improve model accuracy for reviews containing negations.

Key Insight: Model errors often reveal missing linguistic patterns that should be captured in preprocessing. Always examine misclassified examples for recurring patterns.

Made with GAMMA

Case Example 2: Missing Important Sentiment Words

After fitting a Logistic Regression model, inspect the feature coefficients using `model.coef_`. This reveals which words have the strongest influence on predictions.

What You See

Strong Positive:

- excellent → +2.3
- amazing → +2.1
- perfect → +1.8

Strong Negative:

- bad → −1.9
- awful → −1.7

What's Missing

Expected sentiment words like **"fantastic"**, **"terrible"**, and **"disappointing"** don't appear in the feature list at all.

Why? The TF-IDF parameter `min_df` is likely set too high, filtering out rare but important sentiment words.

01

Identify Missing Words

Check vocabulary for expected sentiment terms

02

Diagnose Filtering

Review `min_df` threshold settings

03

Adjust Parameters

Reduce threshold to include rare words



Made with GAMMA

Adjusting Feature Filtering Parameters

Current Setting

`min_df = 10`

Removes words appearing in fewer than 10 documents

Result: "fantastic" (appears 5 times) gets filtered out

Improved Setting

`min_df = 3`

Keeps words appearing in at least 3 documents

Result: Rare but important sentiment words preserved

The parameter `min_df` controls minimum document frequency for keeping a term. While this helps remove noise, setting it too high can accidentally filter out rare but semantically important words.

Trade-off: Lower thresholds include more features but may add noise. Start with `min_df = 3-5` for sentiment analysis, then tune based on validation performance.

Lesson: Feature filtering parameters must balance noise reduction with preserving meaningful rare terms. Always validate that important sentiment words remain in your vocabulary.

Made with GAMMA

Case Example 3: Unstable Cross-Validation Performance

When you run k-fold cross-validation, the accuracy scores vary significantly across folds:

Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
0.88	0.81	0.90	0.79	0.87

The large variation (0.79 to 0.90) indicates your model is sensitive to the specific training data in each fold. This instability suggests overfitting or noisy features.



Problem Sign

Large standard deviation across CV folds (>0.05 suggests instability)



Root Causes

Noisy reviews, insufficient preprocessing, rare words dominating features

Made with GAMMA

Improving Model Stability

Enhance Stopword Removal

Remove more noisy terms that don't contribute to sentiment

Adjust min_df

Increase threshold slightly to remove extremely rare terms causing variance

Add Regularization

In Logistic Regression, reduce `C` parameter for stronger regularization

Regularization Parameter

`LogisticRegression(C=1.0)`

Default value: C=1.0 (no regularization)

For stability: Try C=0.1 or C=0.01

Lower C = stronger regularization = more stable but potentially less accurate

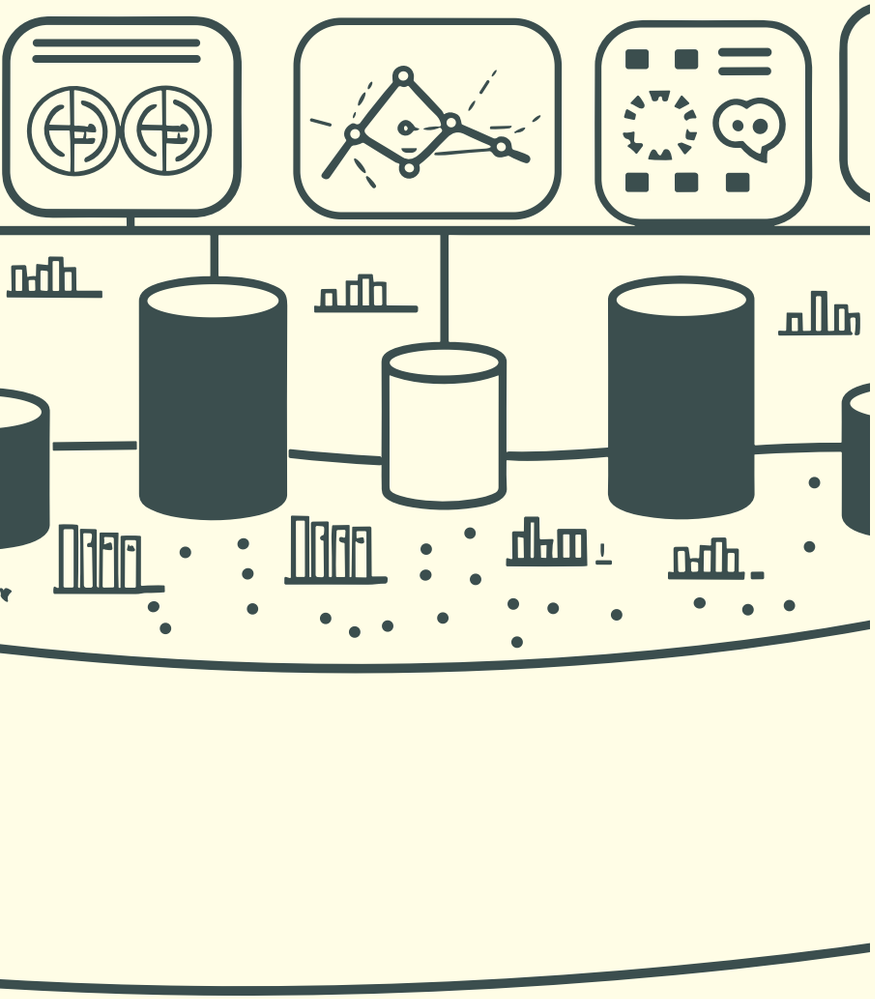
Expected Outcome

After applying these improvements, re-run cross-validation. You should see:

- Smaller variation across folds
- Slightly lower mean accuracy
- Better generalization to new data

Key Insight: Variation in cross-validation scores often indicates feature noise or overfitting. Prioritize stability when deploying models to production.

Made with GAMMA



Scaling in Sentiment Analysis: When It Matters

Different machine learning models behave differently with unscaled features. Understanding these differences helps you decide when scaling is necessary.

Models Requiring Scaling

- Logistic Regression
- Support Vector Machines
- K-Nearest Neighbors
- Neural Networks

These models use **distance-based calculations** where feature magnitude matters

Models Not Requiring Scaling

- Decision Trees
- Random Forest
- Gradient Boosting

These models use **feature threshold splits** where only relative ordering matters

For text-only sentiment analysis using TF-IDF features, scaling is usually **not necessary** because TF-IDF vectors are already L2 normalized. This means values are scaled between documents, preventing any single document from dominating due to length.

Made with GAMMA

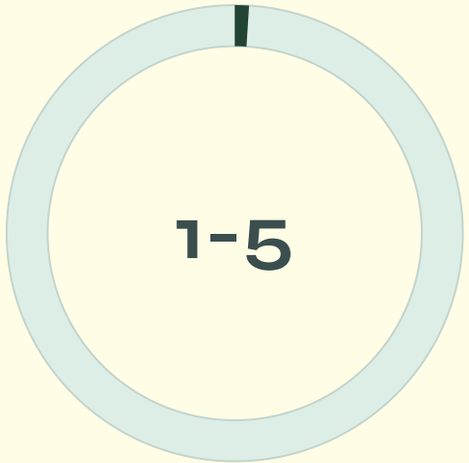
Case Example 4: Scaling Numeric Features

When you extend your sentiment model to include product metadata, scaling becomes important. Consider these feature ranges:



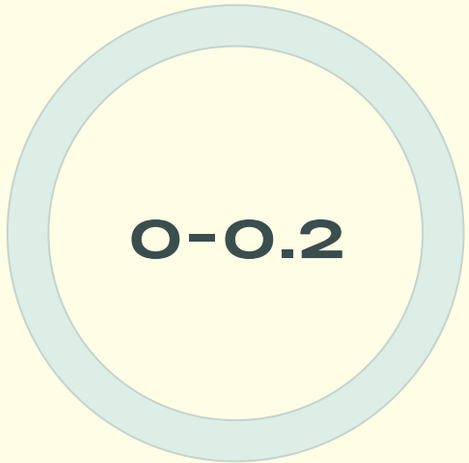
Price

Dollars (range: 49,500)



Rating

Stars (range: 4)



TF-IDF

Normalized weights

The Problem

Without scaling, Logistic Regression may assign excessive importance to **price** simply because its values are much larger in magnitude than other features.

This can be detected by examining `model.coef_` — the price coefficient will be unusually small (or large negative) compared to text feature coefficients.

The Solution

Scale numeric features only, leaving TF-IDF features unchanged since they're already normalized.

`StandardScaler` is best for Logistic Regression and SVM:

```
z = (x − mean) / standard deviation
```

Alternative: `MinMaxScaler` for fixed 0-1 range

Made with GAMMA

Key Diagnostic Lessons

1

Examine Misclassification Patterns

Confusion matrices reveal systematic errors pointing to preprocessing gaps like missing n-grams

2

Inspect Feature Coefficients

Model weights show which terms drive predictions and whether important words were filtered out

3

Monitor Cross-Validation Stability

Large fold variation signals overfitting or noisy features requiring stronger regularization

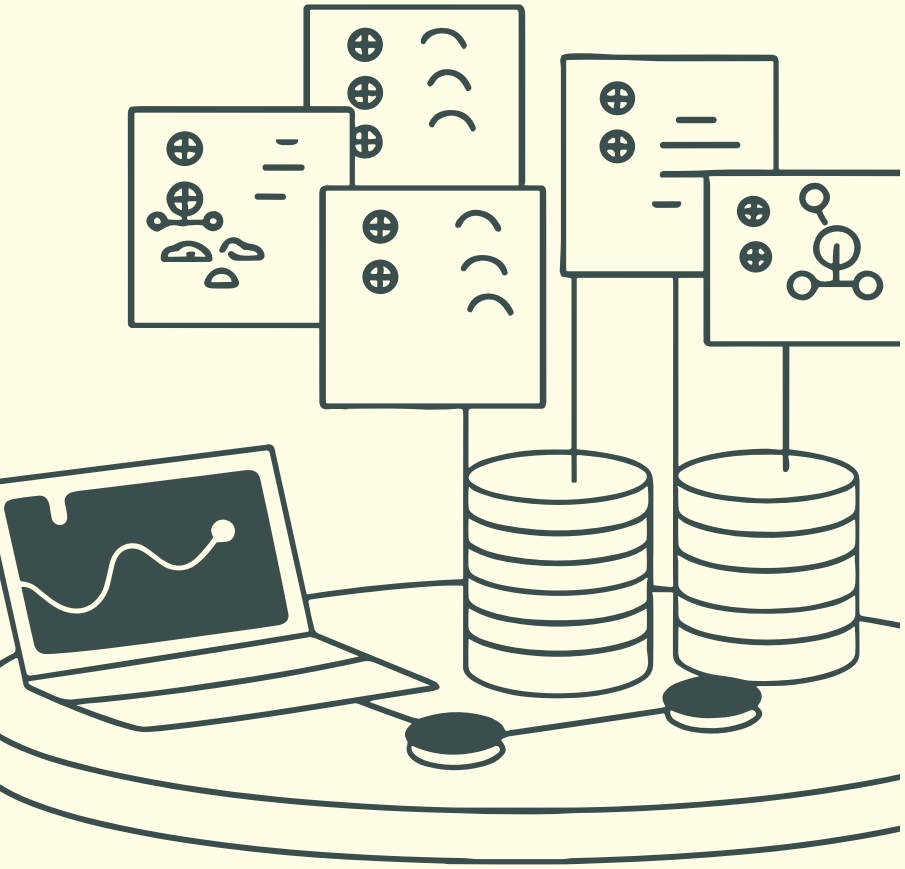
4

Consider Feature Ranges

Combine text features with numeric metadata? Scale the numeric features to prevent magnitude dominance

Remember: Model diagnostics aren't just about accuracy scores. The *patterns* in errors and coefficients provide actionable insights for improving preprocessing, feature engineering, and model configuration.

Made with GAMMA



Cross Validation & Hyperparameter Tuning

Building Reliable Sentiment Analysis Models Through Robust Validation Strategies

Made with GAMMA

Why Train-Test Split Alone Isn't Always Reliable

A single train-test split can produce misleading performance estimates because model accuracy depends heavily on which specific examples end up in the test set. Consider a Logistic Regression model trained on TF-IDF features for sentiment analysis:

0.88	0.80	0.91
Split 1 Accuracy	**Split 2 Accuracy**	**Split 3 Accuracy**
First random split	Second random split	Third random split

These varying results reveal the core problem: **the test set may contain easier or harder examples depending on random sampling**. A single split might give an overly optimistic or pessimistic view of true model performance. The solution? Cross validation provides multiple estimates by systematically rotating which data serves as validation.



Made with GAMMA

Understanding K-Fold Cross Validation

Instead of one train-test split, K-Fold Cross Validation divides the dataset into **K equal parts (folds)**, ensuring every data point is used for validation exactly once. For 5-fold cross validation with our sentiment dataset:



Iteration 1

Train: Fold 2, 3, 4, 5
Validate: Fold 1



Iteration 2

Train: Fold 1, 3, 4, 5
Validate: Fold 2



Continue...

Repeat until each fold serves as validation set once



Final Result

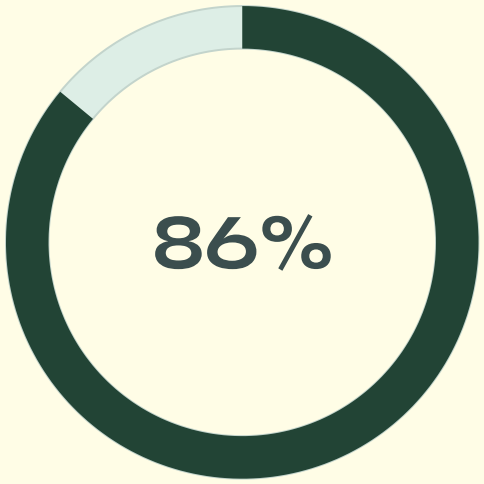
Average performance across all folds

This approach gives a **more reliable estimate of model performance** because it accounts for variability in data sampling. The final model performance is calculated as the average of all fold scores, providing a robust measure of how the model will perform on unseen data.

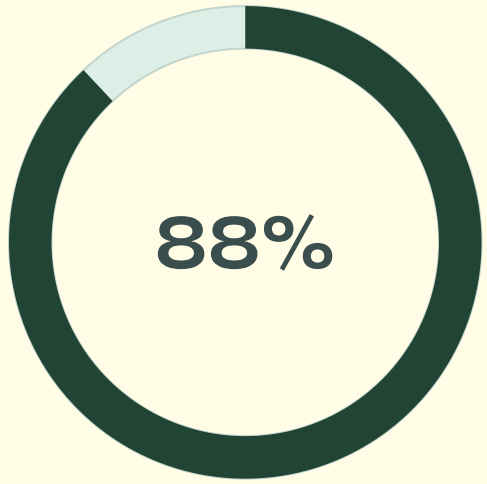
Made with GAMMA

Interpreting Cross Validation Results

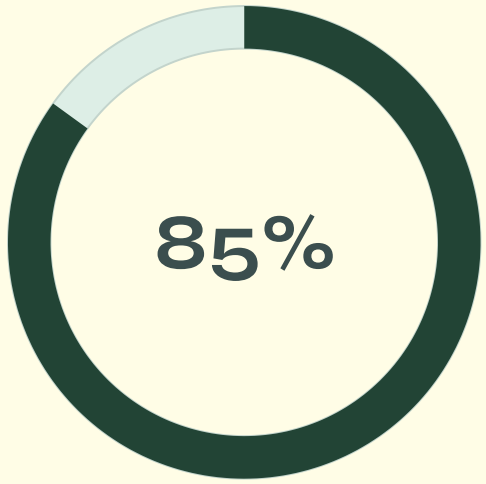
Consider actual results from 5-fold cross validation on our sentiment analysis dataset:



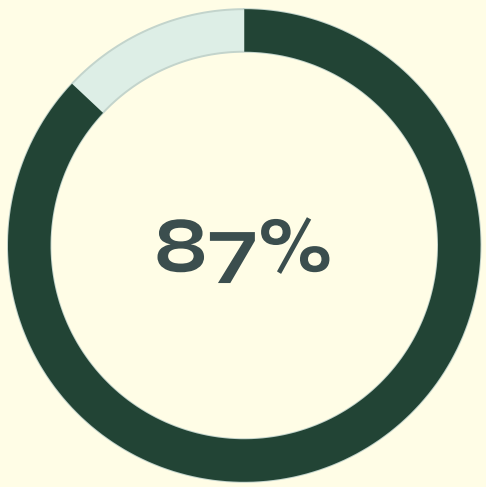
Fold 1



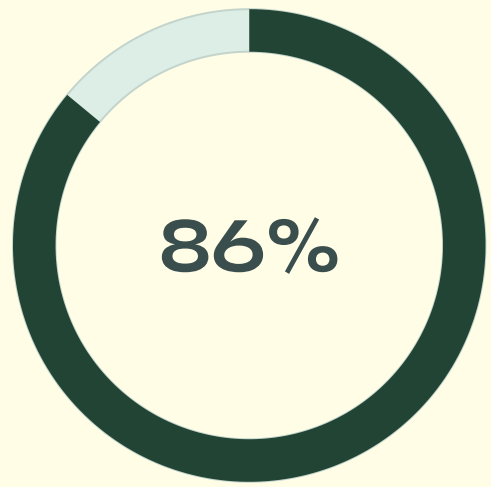
Fold 2



Fold 3



Fold 4



Fold 5

0.864

Mean Accuracy

Expected model performance

0.011

Standard Deviation

Model stability measure

Mean score interpretation: This represents the expected performance of your model on new, unseen data. A mean accuracy of 0.864 suggests your sentiment classifier correctly predicts sentiment about 86.4% of the time.

Standard deviation interpretation: This measures stability across different data splits. A small standard deviation (≈0.011) means the model performs consistently regardless of which fold is used for validation. A large standard deviation would indicate unstable performance that depends heavily on the specific training data.

Made with GAMMA

Stratified K-Fold: Essential for Classification

Two main types of cross validation exist, but **Stratified K-Fold is critical for classification problems** like sentiment analysis.

K-Fold

Randomly splits dataset into K folds without considering class distribution

Stratified K-Fold

Maintains same class distribution in every fold

Why Stratification Matters

Our sentiment dataset has an imbalanced distribution:

- **Positive:** 60% of reviews
- **Neutral:** 25% of reviews
- **Negative:** 15% of reviews

Stratified K-Fold ensures each fold maintains approximately these same proportions, preventing folds that contain very few examples of minority classes (especially negative reviews). This is implemented using `StratifiedKFold` in scikit-learn and is usually preferred for sentiment classification because it provides more realistic performance estimates.

Made with GAMMA

Implementation & Key Takeaways

The implementation of cross validation is straightforward using scikit-learn. The key function is `cross_val_score()`, which handles the entire K-fold process automatically:

```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
scores = cross_val_score(model, X, y, cv=5)

print(scores)
# Output: [0.86, 0.88, 0.85, 0.87, 0.86]
```

Each value in the output array represents the performance of the model on a different validation fold. To summarize results, calculate the mean score for average model performance and standard deviation for stability.

More reliable performance estimates

Average across multiple splits reduces variance from random sampling

Detect unstable models

High standard deviation reveals models sensitive to training data

Compare models fairly

Same validation strategy for all models ensures apples-to-apples comparison

Reduce single split dependence

Eliminates concern about "lucky" or "unlucky" test sets

Important ML Insight: A well-designed validation strategy is critical for building trustworthy machine learning models. Cross validation is the foundation of reliable model evaluation.

Made with GAMMA

Understanding Hyperparameter Tuning

Before diving into hyperparameter tuning, it's essential to distinguish between **parameters** and **hyperparameters**—a fundamental concept often confused by beginners.

Model Parameters

Learned automatically during training

These are the internal weights and coefficients that the model discovers from the data. For example, in Logistic Regression, parameters are the coefficients that multiply each feature to make predictions.

The algorithm determines these values through optimization, not human choice.

Model Hyperparameters

Chosen before training

These control how the model learns and its overall complexity. Examples include regularization strength (C in Logistic Regression), number of trees (n_estimators in Random Forest), and maximum tree depth (max_depth).

These values must be set by the practitioner before training begins.

The goal of hyperparameter tuning is to find the combination of hyperparameters that produces the **best validation performance**, balancing model complexity with generalization ability.

Made with GAMMA

Example: Tuning C in Logistic Regression

The hyperparameter `C` controls **regularization strength** in Logistic Regression. Understanding how C affects model behavior is crucial for effective tuning:

Small C (e.g., 0.01)

Stronger regularization

Pushes coefficients toward zero, creating a simpler model that may underfit

Large C (e.g., 100)

Weaker regularization

Allows coefficients to grow, creating a more complex model that may overfit

Example tuning results using cross validation:

C = 0.01 → CV accuracy = 0.81
C = 0.1 → CV accuracy = 0.85
C = 1 → CV accuracy = 0.86
C = 10 → CV accuracy = 0.86
C = 100 → CV accuracy = 0.83

Interpretation: The best values are around **C = 1 or C = 10**, achieving 86% accuracy. Too much regularization (C=0.01) causes underfitting with only 81% accuracy. Too little regularization (C=100) causes overfitting, dropping to 83% as the model memorizes noise. The sweet spot balances fitting patterns while maintaining generalization.

Made with GAMMA

Example: Tuning Random Forest Hyperparameters

Random Forest has several important hyperparameters. Let's explore how `n_estimators` (number of trees) affects performance:

n_estimators = 50

Accuracy = 0.80

Too few trees, high variance

n_estimators = 100

Accuracy = 0.82

Improvement with more trees

n_estimators = 200

Accuracy = 0.83

Diminishing returns

n_estimators = 500

Accuracy = 0.83

Stabilized performance

Key Insight: Performance improves initially as more trees reduce variance, but eventually stabilizes. Beyond a certain point, increasing model complexity **does not significantly improve performance** while increasing training time and memory usage.

Similar patterns occur with `max_depth` (maximum tree depth). Shallow trees underfit by being too simple. Very deep trees overfit by memorizing training data. The optimal depth balances pattern recognition with generalization to new examples.

Made with GAMMA

Grid Search: Systematic Hyperparameter Optimization

Grid search systematically tests multiple hyperparameter combinations to find the optimal configuration. This approach is essential for efficient model tuning.

1

Define Parameter Grid

Specify ranges to test. For example:

`C = [0.1, 1, 10]`

`solver = ['lbfgs', 'liblinear']`

2

Calculate Combinations

Total models tested = 3 values × 2 solvers = **6 models**

3

Validate Each Model

Each combination evaluated using cross validation

4

Select Best Model

Choose parameters with highest average validation score

Implementation Code

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'C': [0.1, 1, 10],
    'solver': ['lbfgs', 'liblinear']
}

grid = GridSearchCV(model, param_grid, cv=5)
grid.fit(X, y)

print(grid.best_params_)
# Example output: {'C': 1, 'solver': 'lbfgs'}
```

Interpretation: This combination produced the best average validation performance across all folds.

Improves model performance

Finds optimal hyperparameter configuration

Detects overfitting/underfitting

Performance trends reveal model issues

Identifies stable configurations

Similar scores across parameter values indicate robustness

Always combine hyperparameter tuning with cross validation to obtain reliable model performance estimates. Never tune hyperparameters using a single train-test split.

Made with GAMMA

Randomized Search for Hyperparameter Tuning

While Grid Search systematically evaluates every possible combination of hyperparameters, this approach becomes computationally expensive and slow when the parameter space grows large.

The solution is Randomized Search. Instead of testing all combinations, Randomized Search samples a fixed number of random parameter combinations from the search space. This makes it much faster for large parameter spaces.

For example, with Logistic Regression, if we consider parameter ranges like C between 0.001 and 100, and solver = ['lbfgs', 'liblinear'], Randomized Search may test only 10-20 randomly selected combinations instead of all possibilities.

```
RandomizedSearchCV(model, param_distributions, n_iter=20, cv=5)
```

Here, `n_iter` specifies the number of random hyperparameter combinations tested.

An example output for the best parameters might be: `best_params_ = {'C': 3.5, 'solver': 'lbfgs'}`

Randomized Search can often find near-optimal hyperparameters much faster than Grid Search, especially when tuning many parameters.

Grid Search

Use for small parameter spaces

Randomized Search

Use for large search spaces

Made with GAMMA



Why Evaluation Metrics Matter

Evaluating machine learning models correctly is crucial for building reliable systems. Consider a sentiment classifier that reports **90% accuracy**—impressive at first glance. However, if your dataset contains 90% positive reviews, 5% neutral, and 5% negative, a naive model that predicts every review as positive would achieve the same accuracy without learning anything meaningful.

This example reveals why **accuracy alone can be dangerously misleading**, especially with imbalanced datasets. A model might appear successful while completely failing to identify minority classes. This is why we need more sophisticated metrics that provide a complete picture of model performance.

Made with GAMMA

Confusion Matrix: The Foundation

The confusion matrix is the foundation for understanding classification performance. Let's examine a sentiment analysis example where we classify reviews as positive, neutral, or negative.

	Predicted: Pos	Predicted: Neu	Predicted: Neg
Actual: Pos	80 ✓	10	10
Actual: Neu	5	40 ✓	5
Actual: Neg	20	10	60 ✓

True Positives (80)

Positive reviews correctly identified as positive

False Positives (25)

Neutral and negative reviews incorrectly labeled positive

False Negatives (20)

Positive reviews incorrectly labeled neutral or negative

True Negatives (100)

Neutral and negative reviews correctly identified

Key insight: Almost all classification metrics—precision, recall, F1 score, and accuracy itself—are derived from these four fundamental values in the confusion matrix.

Made with GAMMA

Precision, Recall, and F1 Score

1

Precision: Quality of Predictions

Among reviews predicted as positive, how many are actually positive?

Precision = 80 / (80 + 25) = 76%

High precision means few false alarms. Important for spam detection where false positives frustrate users.

2

Recall: Completeness of Detection

Among all actual positive reviews, how many were correctly detected?

Recall = 80 / (80 + 20) = 80%

High recall means catching most positives. Critical for disease detection where missing cases has severe consequences.

3

F1 Score: Balanced Measure

Harmonic mean of precision and recall

F1 = 2 × (0.76 × 0.80) / (0.76 + 0.80) = 78%

Best choice when class imbalance exists and both precision and recall matter equally.

Practical guidance: Choose precision when false positives are costly (spam filters), recall when false negatives are dangerous (medical diagnosis), and F1 score for imbalanced datasets with equal concern for both errors.

Made with GAMMA

Evaluation Metrics for Regression

Regression tasks predict continuous values rather than discrete classes. For example, predicting product ratings on a 1–5 star scale from review text requires different evaluation metrics than classification.

1-5

Target Range

Product ratings from one to five stars

0.5

Example Error

Actual: 4 stars, Predicted: 3.5 stars

Mean Absolute Error (MAE)

MAE calculates the average absolute difference between predicted and actual values. It treats all errors equally regardless of direction.

```
MAE = (|4-3.5| + |2-2.2| + |5-4.8| + ...) / N
```

Interpretation: If MAE = 0.5 stars, predictions are off by half a star on average. Easy to understand and interpret in original units.

Made with GAMMA

Mean Squared Error and RMSE

Mean Squared Error (MSE)

MSE squares errors before averaging them, giving **higher penalty to large errors**.

$$MSE = ((4-3.5)^2 + (2-2.2)^2 + (5-4.8)^2 + \dots) / N$$

Squaring amplifies large errors: a 2-star mistake counts four times worse than a 1-star mistake.

Root Mean Squared Error (RMSE)

RMSE takes the square root of MSE, returning to original units.

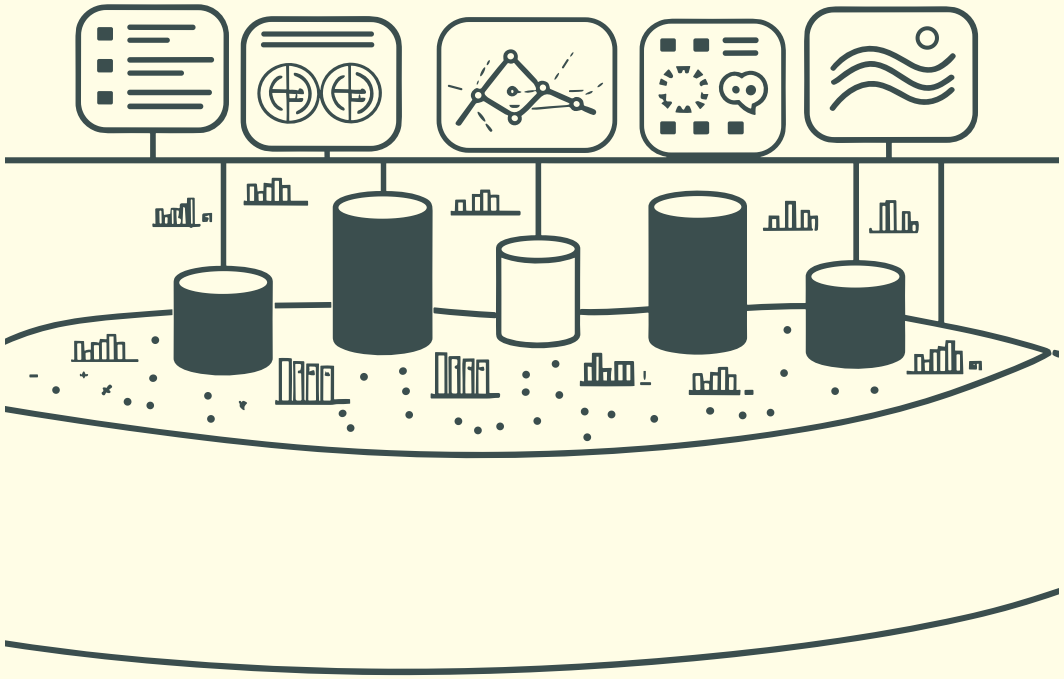
$$RMSE = \sqrt{MSE}$$

Key advantage: RMSE = 0.6 stars means predictions are off by about 0.6 rating points on average—intuitive interpretation.

Why RMSE matters: It's expressed in the same units as your target variable (stars), making it directly interpretable. RMSE penalizes large errors more than MAE, which is valuable when large mistakes are particularly costly. Most regression competitions use RMSE or MSE as the primary metric.

Made with GAMMA

Choosing the Right Evaluation Metric





Sentiment Classification

Preferred metric: F1 Score

Sentiment datasets typically have class imbalance (many positive, few negative reviews). F1 score balances precision and recall, ensuring the model identifies minority classes while maintaining quality predictions.

Rating Prediction

Preferred metric: RMSE

Predicting continuous ratings requires measuring magnitude of errors. RMSE reflects how far predictions deviate from actual values in interpretable units (rating points), penalizing large mistakes appropriately.

Golden rule: Evaluation metrics must align with your business objective. Choose metrics that directly reflect what success means in your specific application, not what's popular or convenient.

Made with GAMMA

Understanding Overfitting

Overfitting occurs when a model **memorizes the training data rather than learning general patterns**. The model becomes too specialized to the training set and fails to generalize to new, unseen data.

97%

Training Accuracy

Model performs extremely well on training data it has seen

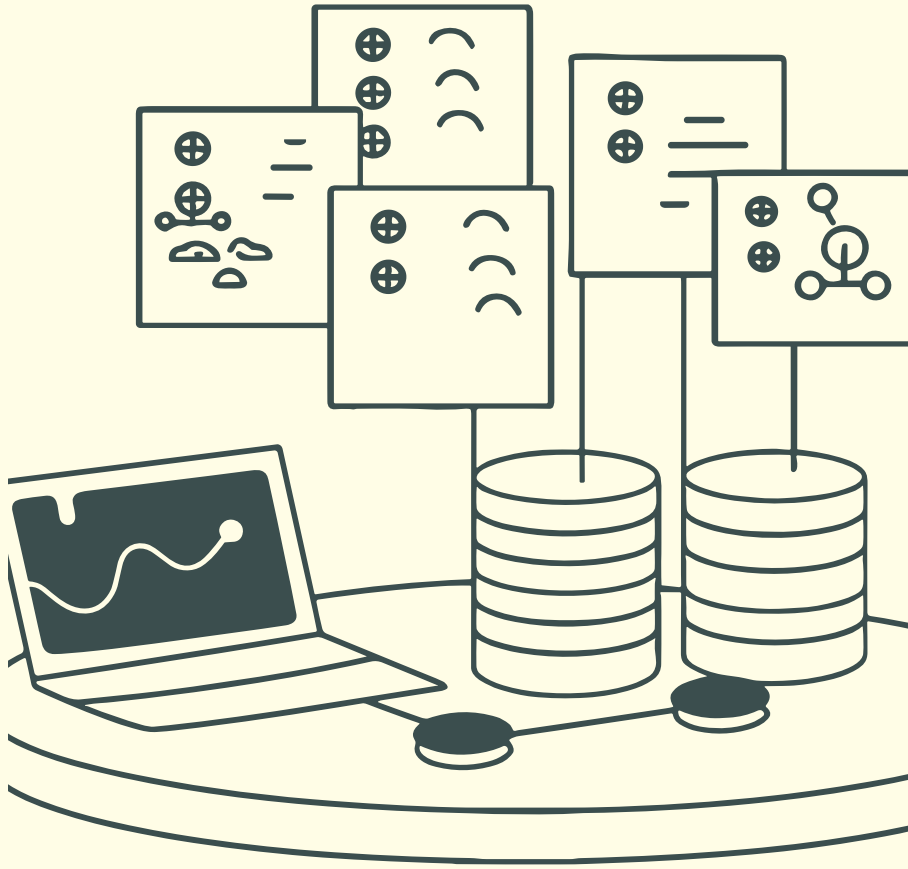
82%

Test Accuracy

Poor performance on new data it hasn't encountered before

What's happening: The model has learned noise, outliers, and specific patterns in the training data that don't generalize. It's like memorizing answers to practice questions instead of understanding the underlying concepts.

Warning signs: Large gap between training and test performance, excellent training metrics but poor real-world results, model complexity far exceeding what the data justifies.



Made with GAMMA

Underfitting vs Good Fit vs Overfitting

Understanding the spectrum of model fit helps diagnose problems and guide improvements.

Underfitting

Train: 70%
Test: 68%

Model too simple to capture underlying patterns. Both training and test performance are poor. Solution: increase model complexity or add features.

Good Fit ✓

Train: 88%
Test: 86%

Model generalizes well with minimal gap between training and test. Captures patterns without memorizing noise. This is the target zone.

Overfitting

Train: 97%
Test: 82%

Model too complex, memorizing training data. Large performance gap indicates poor generalization. Solution: reduce complexity or add regularization.

Key insight: The goal is achieving a balance where model complexity matches the complexity of the underlying patterns in your data, not maximizing training performance.

Made with GAMMA

Bias–Variance Tradeoff

Model errors decompose into two components. Understanding this tradeoff helps diagnose and fix model problems.

Bias: Error from Simplistic Assumptions

High bias means the model makes overly simplistic assumptions about the data.

Example: Using a linear model to fit a complex nonlinear relationship. The model cannot capture the true patterns regardless of training data size.

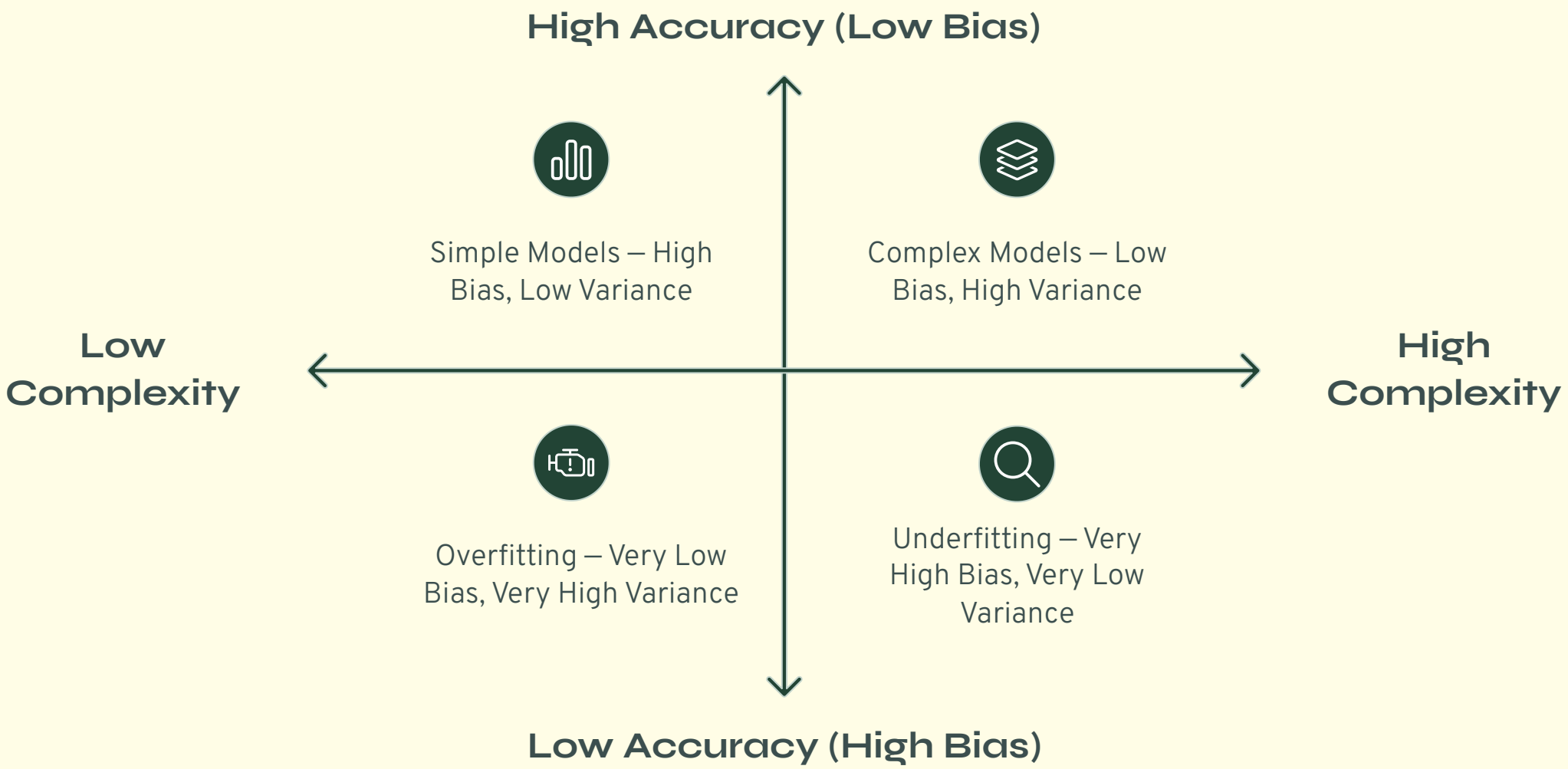
Results in **underfitting**—poor performance on both training and test data.

Variance: Error from Sensitivity to Training Data

High variance means the model is too sensitive to fluctuations in the training data.

Example: A very deep decision tree that creates rules for every training example. Small changes in training data produce completely different models.

Results in **overfitting**—excellent training performance but poor test performance.



Made with GAMMA

Practical Methods to Reduce Overfitting

01

Regularization

Add penalties to model complexity. In scikit-learn, adjust parameters like `LogisticRegression(C=1.0)` where smaller C values increase regularization strength.

02

Cross Validation

Use k-fold cross validation to detect unstable models. If performance varies dramatically across folds, the model likely overfits to specific data subsets.

03

Simpler Models

Reduce model complexity through hyperparameter tuning. For decision trees, limit depth. For neural networks, reduce layers or neurons.

04

More Training Data

Additional data helps reduce variance by providing more examples of underlying patterns, making it harder for models to memorize noise.

05

Improved Feature Engineering

In sentiment analysis, adjust TF-IDF parameters like `min_df` to limit extremely rare words, use appropriate n-gram ranges, and remove irrelevant features.

Final takeaway: Successful machine learning requires balancing model complexity, preprocessing quality, and proper evaluation metrics. Regularization, cross validation, and thoughtful feature engineering are essential tools in your toolkit.

Made with GAMMA

Ensemble Modeling for Sentiment Analysis

Ensemble learning combines multiple machine learning models to create more robust and accurate predictions than any single model alone. By leveraging the strengths of different algorithms, ensemble methods can capture diverse patterns in data that individual models might miss. In sentiment analysis, where text interpretation can vary significantly, combining models becomes particularly valuable.

When building effective ensembles, we don't simply combine all available models. Instead, we follow a systematic approach: first perform cross-validation and hyperparameter tuning on each candidate model, then select the two best performers based on evaluation metrics like accuracy and F1 score. This ensures we're combining high-quality models rather than diluting performance with weaker ones.



Made with GAMMA

Complete Pipeline for Sentiment Analysis

The sentiment analysis pipeline transforms raw text into actionable predictions through systematic processing steps:

1

Text Data

Raw reviews, tweets, or customer feedback

2

Text Preprocessing

Cleaning, normalization, and tokenization

3

TF-IDF Vectorization

Convert text to numerical features

4

Model Training

Train individual ML models

5

Ensemble Model

Combine best performers

TF-IDF (Term Frequency-Inverse Document Frequency) converts text into numerical vectors by measuring how important a word is to a document relative to the entire corpus. Words that appear frequently in a document but rarely elsewhere receive high weights, making them valuable features for sentiment classification.

Made with GAMMA

Selecting the Best Performing Models

Cross-validation provides reliable performance estimates for each candidate model. We examine multiple metrics including accuracy, precision, recall, and F1 score to identify the strongest performers. In text classification tasks, certain models consistently excel due to their ability to handle high-dimensional sparse data.

Based on cross-validation results, we select the two best models for ensemble construction. This approach ensures we're combining high-performing algorithms rather than including weaker models that might degrade overall performance.

Model	CV Accuracy	F1 Score
Logistic Regression	0.86	0.85
Linear SVM	**0.87**	**0.86**
Random Forest	0.82	0.81

Why Logistic Regression and SVM? These models excel with TF-IDF features because they handle sparse, high-dimensional data effectively and provide probability estimates for soft voting ensembles.

Made with GAMMA

Understanding Classification Errors

In classification tasks, we analyze misclassified samples rather than residuals. Different models make different mistakes because they capture different patterns in the data. This diversity is precisely what makes ensemble methods powerful—when one model errs, another might be correct.

Review	Actual	Logistic Regression	SVM
"Great product"	Positive	Positive	Positive
"Not good at all"	Negative	Positive	Negative
"Average quality"	Neutral	Neutral	Positive

The example shows complementary error patterns: Logistic Regression misclassifies negation phrases while SVM struggles with neutral reviews. An ensemble can leverage their strengths—when both agree, we're confident; when they disagree, we can use probability averaging to make better decisions.

Made with GAMMA

Analyzing Difficult Classification Cases

Negation Phrases

Words like "not," "never," "nothing" reverse sentiment meaning. Models must understand context and word order to handle these correctly.

Mixed Sentiment

Sentences containing both positive and negative aspects require understanding which dominates or recognizing multi-faceted opinions.

Neutral Reviews

Objective descriptions without clear sentiment are challenging. Models need to distinguish between neutral and ambiguous sentiment.



Studying classification errors reveals patterns in difficult cases, similar to how residual analysis in regression identifies problematic data points. This error-focused approach helps improve preprocessing steps, feature engineering, or model selection. For example, identifying frequent negation errors might prompt adding negation handling to preprocessing, while mixed sentiment cases suggest needing more sophisticated feature representations.

Made with GAMMA

Soft Voting: Combining Probabilities

Soft voting combines prediction probabilities from multiple models rather than just final class predictions. Each model outputs a probability distribution across all classes, and we average these probabilities to make the final prediction. This approach leverages the confidence levels of each model, giving more weight to predictions where models are certain.

The formula for soft voting with two models is straightforward:

$$P_{final}(c) = \frac{P_{model1}(c) + P_{model2}(c)}{2}$$

where c represents each class and P represents probability.

Model	Positive	Neutral	Negative
Logistic Regression	0.55	0.20	0.25
SVM	0.30	0.10	0.60
Average (Ensemble)	**0.425**	**0.15**	**0.425**

Both classes tie at 0.425, but real examples typically show clear winners. The ensemble makes predictions based on the highest average probability.

Made with GAMMA

Implementing Ensemble with scikit-learn

Scikit-learn provides the `VotingClassifier` class to easily create ensemble models. The implementation is straightforward—specify the individual estimators and the voting strategy.

```
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC

# Train individual models first
lr_model = LogisticRegression()
svm_model = LinearSVC()

# Create ensemble with soft voting
ensemble_model = VotingClassifier(
    estimators=[('lr', lr_model), ('svm', svm_model)],
    voting='soft'  # 'soft' for probability averaging, 'hard' for majority vote
)

# Fit ensemble on training data
ensemble_model.fit(X_train, y_train)

# Make predictions
predictions = ensemble_model.predict(X_test)
```

Soft Voting

Averages predicted probabilities. Requires models to support probability estimation. Generally performs better when models are well-calibrated.

Hard Voting

Takes majority vote of predicted classes. Works with any classifier. May be less effective when models have varying confidence levels.

Made with GAMMA

Weighted Ensemble Strategy

Sometimes models have different performance levels, making equal weighting suboptimal. Weighted ensembles assign different importance to each model based on their validation performance, giving more influence to better-performing models.

01

Evaluate Individual Models

Calculate F1 scores or accuracy on validation set for each model

02

Calculate Weights

Convert performance scores to normalized weights (higher scores get larger weights)

03

Apply Weighted Averaging

Multiply each model's probabilities by its weight and sum

Example Weighted Combination

Suppose SVM has F1=0.86 and Logistic Regression has F1=0.85. We might assign weights:

- Logistic Regression: 0.45
- SVM: 0.55

The final probability calculation becomes:

$$P_{final} = 0.45 \times P_{LR} + 0.55 \times P_{SVM}$$

Implementation Note

Scikit-learn's `VotingClassifier` supports weighted ensembles through the `weights` parameter:

```
ensemble_model = VotingClassifier(
 estimators=[('lr', lr_model), ('svm', svm_model)],
 voting='soft',
 weights=[0.45, 0.55]
)
```

Made with GAMMA

Error-Focused Ensemble Approach

An advanced ensemble strategy trains subsequent models specifically on difficult cases that previous models misclassified. This creates a cascade where each model learns from the mistakes of its predecessor, focusing on progressively harder examples.



Train Initial Model

Fit first model (Model 1) on complete training data and evaluate performance

Identify Misclassified Samples

Find training examples where Model 1 made incorrect predictions—these represent difficult cases

Train Second Model

Fit Model 2 specifically on misclassified samples from Model 1

Combine Predictions

Use ensemble logic to leverage both models' strengths

This approach helps improve predictions for complex sentences and edge cases. The second model becomes a specialist in handling problematic patterns that the first model missed. When making predictions, we can use confidence scores to determine which model to trust for each sample, or combine predictions through weighted voting.

Made with GAMMA

Performance Improvement Results

Model	F1 Score
Logistic Regression	0.85
Linear SVM	0.86
Ensemble Model	**0.88**

The ensemble achieves **2-3% improvement** over individual models, which can be significant in production systems.

Why Ensembles Outperform

Ensemble models combine complementary strengths: different algorithms capture different patterns, errors are uncorrelated, and probability averaging smooths predictions.

Diverse Patterns

Different models capture different data patterns and relationships

Error Analysis

Studying misclassifications reveals difficult cases and improvement opportunities

Robust Accuracy

Ensemble predictions are more stable and accurate than individual models

Real-World Impact: Ensemble methods are widely used in machine learning competitions (Kaggle) and production systems because they consistently deliver superior performance with minimal additional complexity.

Made with GAMMA